

Habib University



Dhanani School of Science and Engineering

Digital Logic and Design

(EE-172L / CS-130L)

Project Documentation

Two Player Tank Game

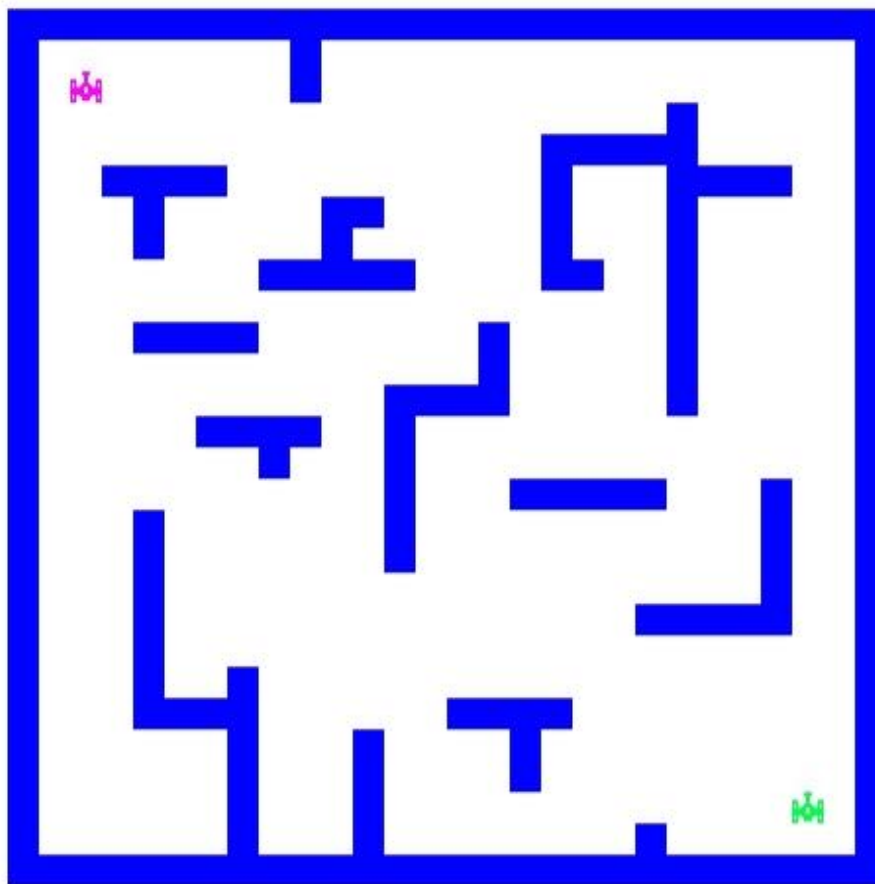
Sameer Pervez

Muhammad Aqib Khan

Mohammad Hasan Tariq

Yabudullah Ahmed Bakhtiar

Two Player Tank Game



This image contains the initial version of the game, the current one can be seen on the last pages of this report

Game Features:

The game consists of two tanks for player 1 and player 2 respectively. The tanks are controlled simultaneously by two users using keyboard input. The main objective of the game is to win either by collecting the crown when it has the jewel or eliminating the other player's tank before they collect the crown. The tanks move in a map which consists of many walls, tanks can take cover behind walls to avoid getting hit by the shell.

Playing Instructions:

Player 1 and Player 2 can move in 4 directions using the designated keyboard keys.

General Controls:

Space => END SCREEN to START SCREEN

Enter => START SCREEN to GAME SCREEN

Player 1 Controls:

Up Arrow => Face up / Move up

Down Arrow => Face down / Move down

Left Arrow => Face left / Move Left

Right Arrow => Face right / Move right

RShift => Shoot

Player 2 Controls:

W => Face up / Move up

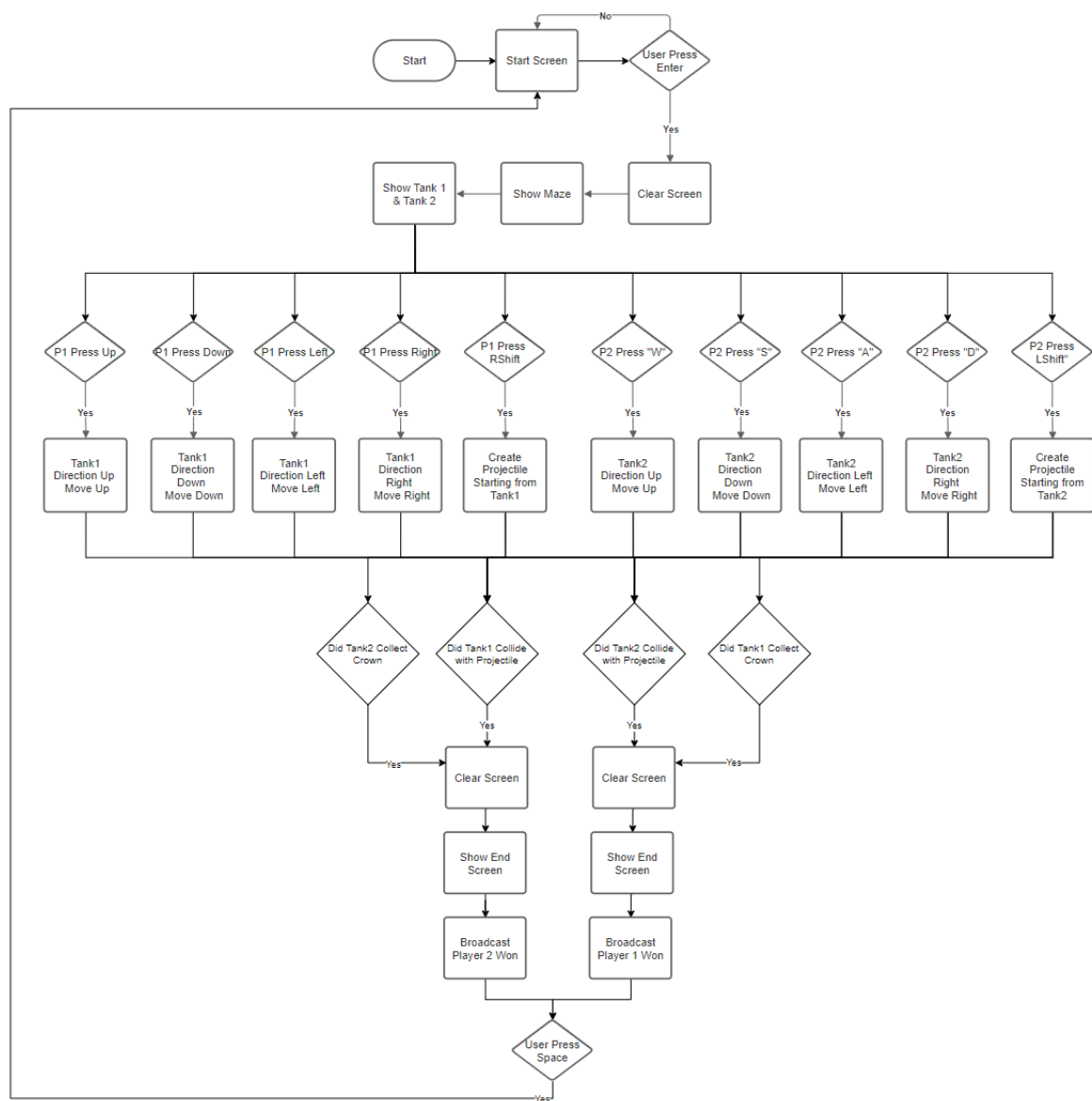
S => Face down / Move down

A => Face left / Move Left

D => Face right / Move right

LShift => Shoot

User Flow Diagram:



Input Block:

Input Block will manage the input stream from the keyboard and translate it to store the currently pressed keys along with detecting unpressed state.

Control Block:

Control block will manage all of the game's finite state machines controlling start screen, game screen, direction, movement, shooting, collision and end screen.

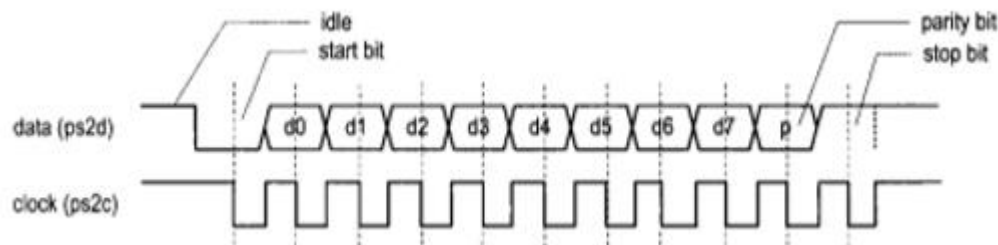
Output Block:

Output block will manage the data from the control block and display the bitmaps of the map, both tanks and projectile. It will consist of modules such as h_sync, v_sync, pixel_gen and vga_controller.

Input Block Implementation:

The PS2 controller of the BASYS-3 board is used to implement the keyboard inputs.

The keyboard sends key data in bitstream which is stored in *kdata* bus which is stored into data cur based on the keyboard clock *kclk* and stop bit. With every new flag the current data from the data cur register is stored into *data_prev* and the keyboard data is stored into data cur. The register led holds both *data_prev* and *data_cur*, if *data_prev* = F0 it means the key in *data_cur* has been released, otherwise the key in *data_cur* is pressed.

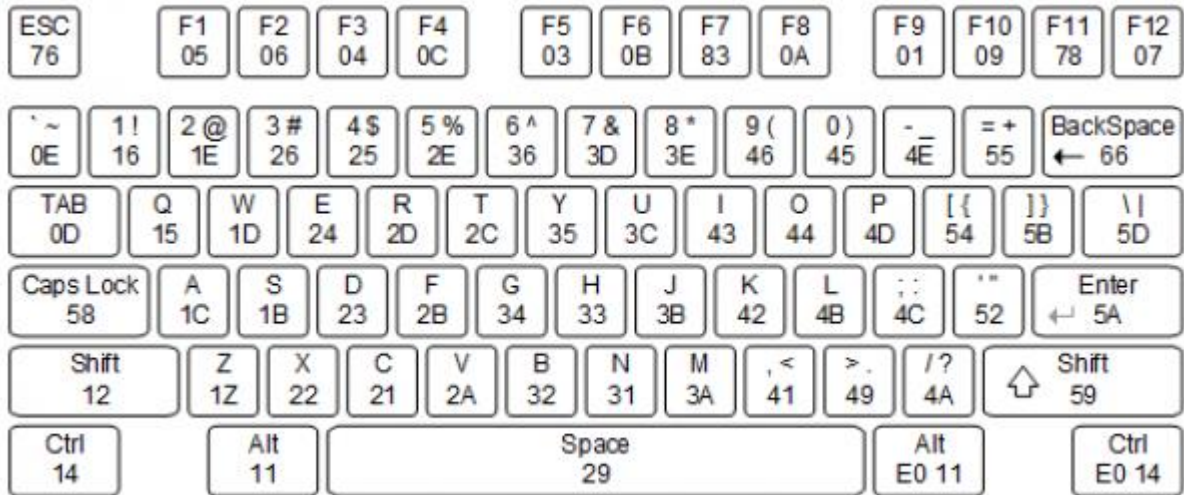


Pin Configuration:

INPUT	PIN	INPUT	PIN
kclk	C17	Kdata	B17

IO Details:

KEY	HEX	KEY	HEX
W	1D	↑	75
A	1C	←	6B
S	1B	↓	72
D	23	→	74
LSHIFT	12	RSHIFT	59



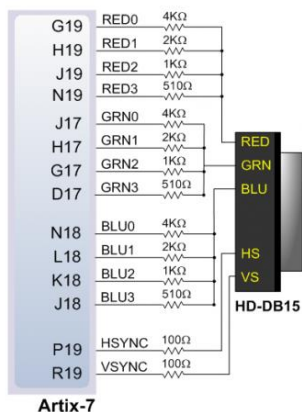
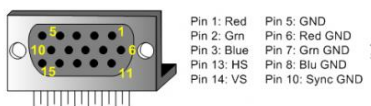
Output Block Implementation:

The clock divider module generates a clock of 25MHz which is sent to h_sync and v_sync modules. The h_sync module increases the h_count which corresponds to the pixel_x (the horizontal pixel location) on the screen and v_sync module increases the v_count when h_count reaches its maximum value (right most corner of the screen). This is how all the pixels of the screen are accessed.

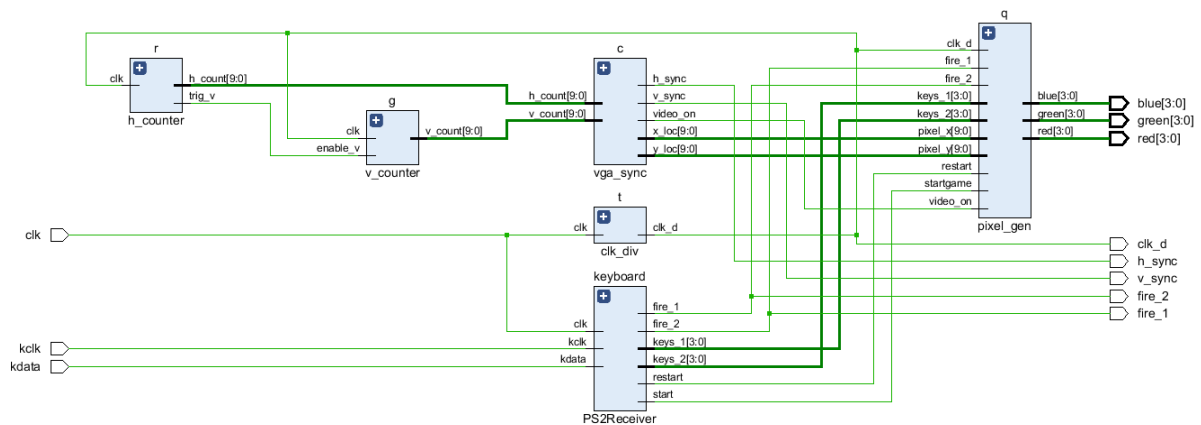
The pixel_gen module assigns an RGB value to every pixel of the screen which is called by a top-level module vga_controller (the main display module) which generates and updates the output of the screen coming from the Control Block.

The bitmaps for tank1, tank2 and map are called also by pixel_gen module to receive where to print what pixels.

Pin Configuration:



Top-Level Module Block Diagram:

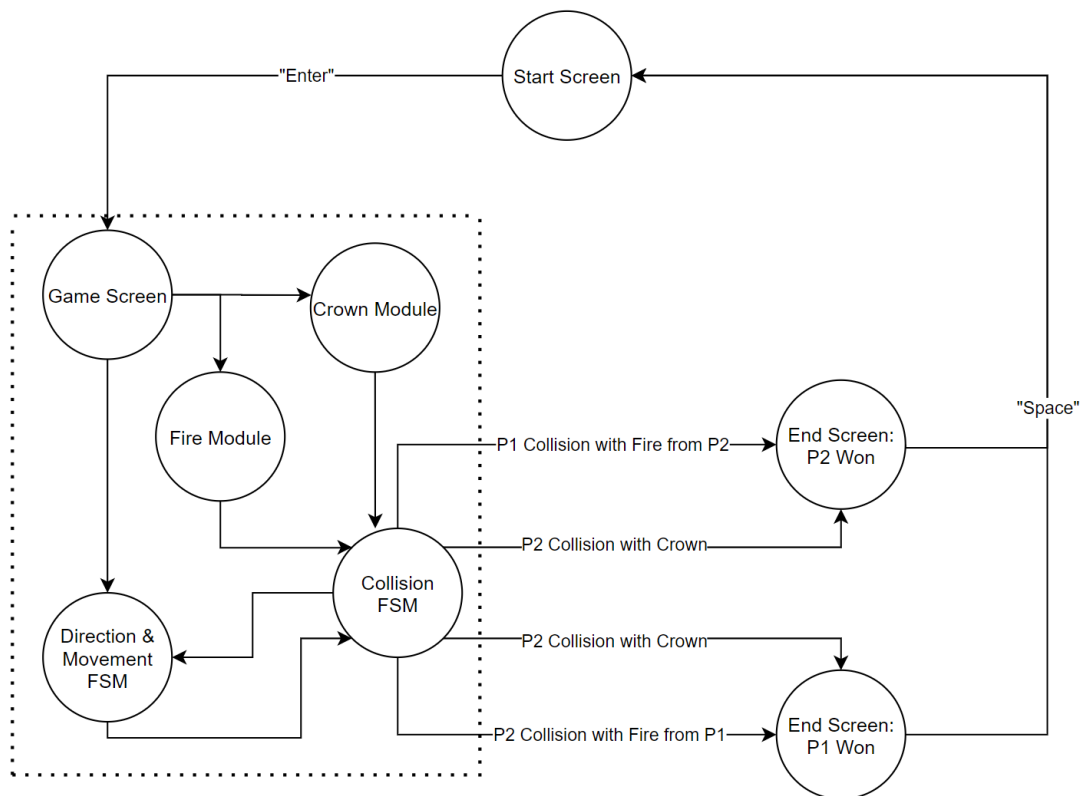


Control Block Implementation:

The main game's FSM (Mealy Machine) has four primary states:

1. Start Screen
2. Game Screen
3. End Screen: P1 wins
4. End Screen: P2 wins

Top level Control Block:



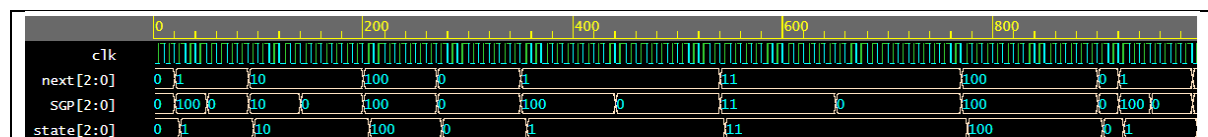
Working:

This explains the core mechanics of the game. The game begins at the “start” state. On the press of “enter” key, it switches to “game” state where the actual game runs. In the game, if tank 1 collides with tank 2’s fire, tank 2 wins, and vice versa. If either tank collides with the crown while the gem is appearing, that player wins. Upon a player winning, the game switches to that specific player’s win game state where the winning message is displayed. Here, the press of “space” key will switch the game back to “start” state, and everything is repeated.

Implementation:

FSM	Testbench
<pre>module FSM_screen (clk, SGP, state, next); input clk; input [2:0] SGP; output [2:0] state, next; reg [2:0] state = 2'b0; reg [2:0] next = 2'b0; parameter start = 3'b0; parameter game = 3'b1; parameter p1 = 3'b10; parameter p2 = 3'b11; parameter start_space = 3'b100; always @(posedge clk) begin state <= next; end always @(SGP or state) begin case (SGP[2]) 1'b0: if (state == start_space) next = start; 1'b1: if (state == start) next = game; else if (state == p1 state == p2) next = start_space; endcase case (SGP[1:0]) 2'b10: if (state == game) next = p1; 2'b11: if (state == game) next = p2; endcase end endmodule</pre>	<pre>module test(); reg clk; FSM_screen t (clk,SGP,state, next); initial begin clk <= 0; SGP <= 0; end always #5 clk = ~clk; always begin #20 SGP = 3'b100; #30 SGP = 3'b0; #40 SGP = 3'b10; #50 SGP = 3'b0; #60 SGP = 3'b100; #70 SGP = 3'b0; #80 SGP = 3'b100; #90 SGP = 3'b0; #100 SGP = 3'b11; #110 SGP = 3'b0; #120 SGP = 3'b100; #130 SGP = 3'b0; end initial begin \$dumpfile("dump.vcd"); \$dumpvars(1,test); #1000 \$finish; end endmodule</pre>

Timing Diagram:

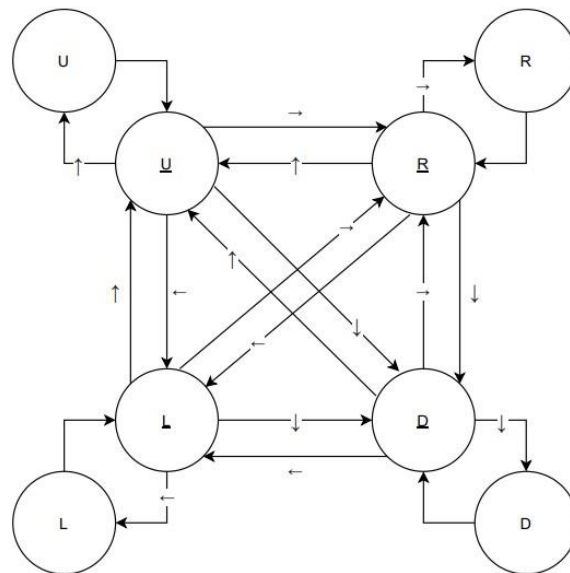


Game Screen:

The 'Game Screen' state has been created which consists of multiple modules that handle the playable map area, two tanks for both players and their movement with direction, firing, crown collection and collision with walls as well as fire projectile.

Movement & Direction FSM:

The movement and direction of the tanks are controlled using a separate FSM



States		Inputs	
U	Up Movement	↑	Up
D	Down Movement	↓	Down
L	Left Movement	←	Left
R	Right Movement	→	Right
<u>U</u>	Up Direction Stopped		
<u>D</u>	Down Direction Stopped		
<u>L</u>	Left Direction Stopped		
<u>R</u>	Right Direction Stopped		

where the bar underneath the letter represents the direction states.

Working:

There are 4 states in which the tank is moving for each of the four directions (up down, left, and right) and there are 4 more in which the tank is standing in each of the 4 directions. The tank switches its direction on the press of an arrow key if it is not aligned with the desired direction of the pressed key and starts moving when if the direction of the key pressed aligns with the state in which the tank is in. This ensures that the tank only moves in the direction if it is facing in the same direction.

State Table:

Current State	Previous State			Rest 0000			Up 0001			Down 0010			Left 0100			Right 1000		
	A	B	C	A	B	C	A	B	C	A	B	C	A	B	C	A	B	C
<u>U</u>	0	0	0	0	0	1	0	0	1	0	1	0	1	0	0	1	1	0
U	0	0	1	0	0	1	0	0	1	0	1	0	1	0	0	1	1	0
<u>D</u>	0	1	0	0	1	0	0	0	0	0	1	1	1	0	0	1	1	0
D	0	1	1	0	1	0	0	0	0	0	1	1	1	0	0	1	1	0
<u>L</u>	1	0	0	1	0	0	0	0	0	0	1	0	1	0	1	1	1	0
L	1	0	1	1	0	0	0	0	0	0	1	0	1	0	1	1	1	0
<u>R</u>	1	1	0	1	1	0	0	0	0	0	1	0	1	0	0	1	1	1
R	1	1	1	1	1	0	0	0	0	0	1	0	1	0	0	1	1	1

State Equations:

$$A_{(t+1)} = AU'D'R'L' + R'L'D'U' + RL'D'U'$$

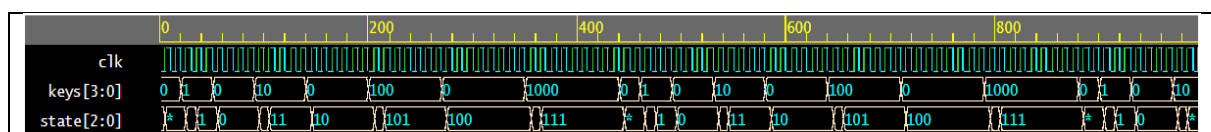
$$B_{(t+1)} = BU'D'R'L' + R'L'DU' + RL'D'U'$$

$$C_{(t+1)} = A'B'R'L'D'U + A'BR'L'DU' + AB'R'LD'U' + ABRL'D'U'$$

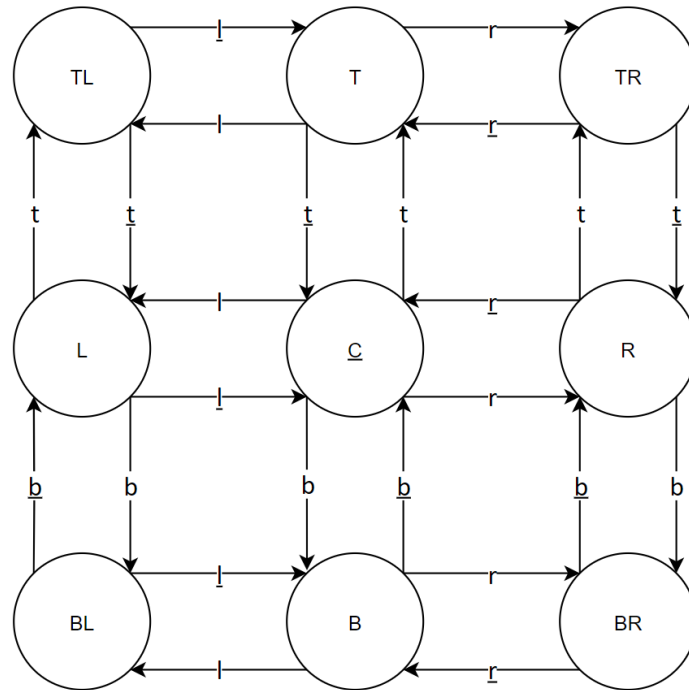
Implementation:

FSM	Testbench
<pre> module FSM_move(clk, keys, state); input clk; input [3:0] keys; output [2:0] state; reg [2:0] state = 0; reg [2:0] next; parameter up = 3'b0; parameter up_ = 3'b1; parameter down = 3'b10; parameter down_ = 3'b11; parameter left = 3'b100; parameter left_ = 3'b101; parameter right = 3'b110; parameter right_ = 3'b111; always @(posedge clk) begin state <= next; end always @(keys or state) begin case (keys) 4'b0: next[0] = 0; 4'b1: if (state == up) next = up_; else if (state != up_) next = up; 4'b10: if (state == down) next = down_; else if (state != down_) next = down; 4'b100: if (state == left) next = left_; else if (state != left_) next = left; 4'b1000: if (state == right) next = right_; else if (state != right_) next = right; endcase end endmodule </pre>	<pre> module test(); reg clk; reg [3:0] keys; wire [2:0] state; FSM_move t (clk,keys,state); initial begin clk <= 0; keys <= 0; end always #5 clk = ~clk; always begin #20 keys = 4'b0001; #30 keys = 0; #40 keys = 4'b0010; #50 keys = 0; #60 keys = 4'b0100; #70 keys = 0; #80 keys = 4'b1000; #90 keys = 0; end initial begin \$dumpfile("dump.vcd"); \$dumpvars(1,test); #1000 \$finish; end endmodule </pre>

Timing Diagram:



Collision FSM:



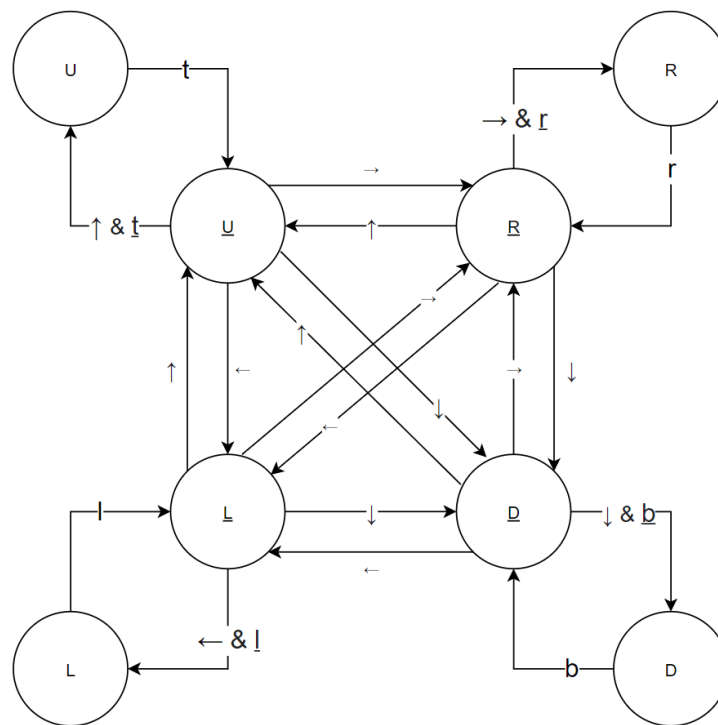
States		Inputs	
<u>C</u>	No Collision	t	Top Collision
T	Top Collision	b	Bottom Collision
B	Bottom Collision	l	Left Collision
L	Left Collision	r	Right Collision
R	Right Collision		
TR	Top & Right Collision		
TL	Top & Left Collision		
BR	Bottom & Right Collision		
BL	Bottom & Left Collision		

where the bar underneath the letter represents the complement of input/state.

Working:

The game detects collision on each side of the tank, and turns the relevant flag on (e.g. collision of an object with top of tank will turn flag “t” to true). Based on the four collision flags, the FSM sets the state of collision.

Movement & Direction FSM with Collision Input:



States		Inputs	
U	Up Movement	↑	Up
D	Down Movement	↓	Down
L	Left Movement	←	Left
R	Right Movement	→	Right
<u>U</u>	Up Direction Stopped	t	Top Collision
<u>D</u>	Down Direction Stopped	b	Bottom Collision
<u>L</u>	Left Direction Stopped	l	Left Collision
<u>R</u>	Right Direction Stopped	r	Right Collision

where the bar underneath the letter represents the complement of input/state.

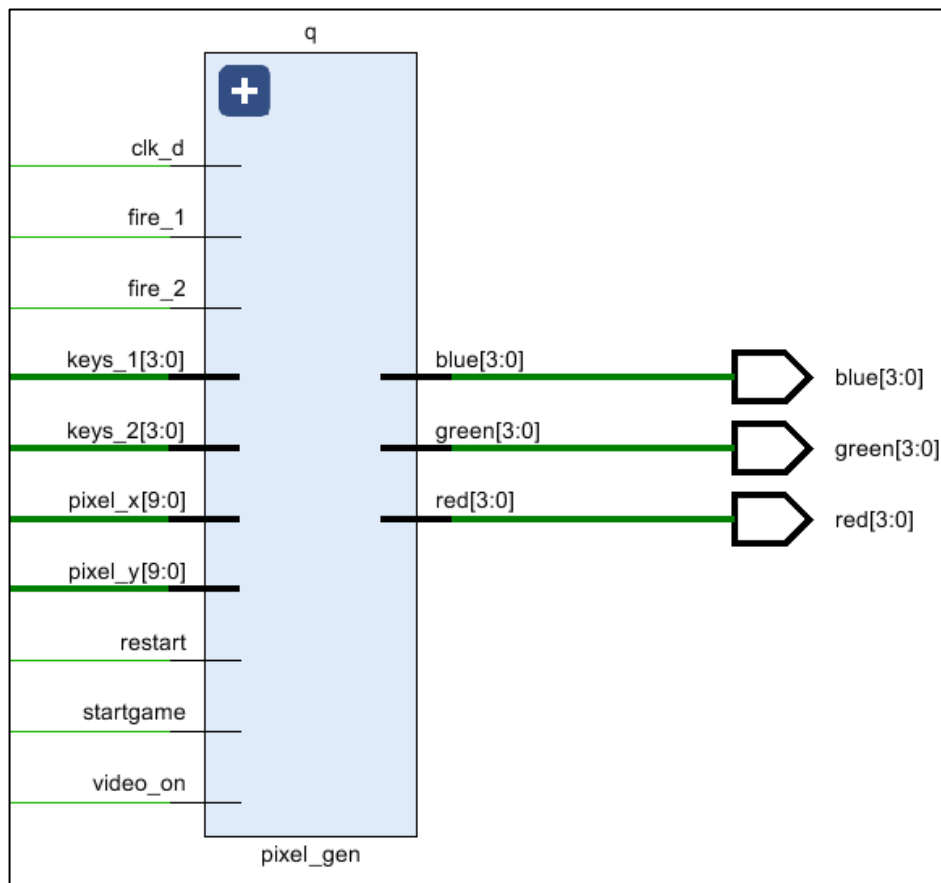
Working:

This is the combination of movement FSM and collision FSM. In the movement FSM, the tank will not switch to moving state if the state is such that collision is happening on that side.

Pixel Gen:

Pixel Gen colorizes the pixels according to the information received by the control block of the game.

Block Diagram:

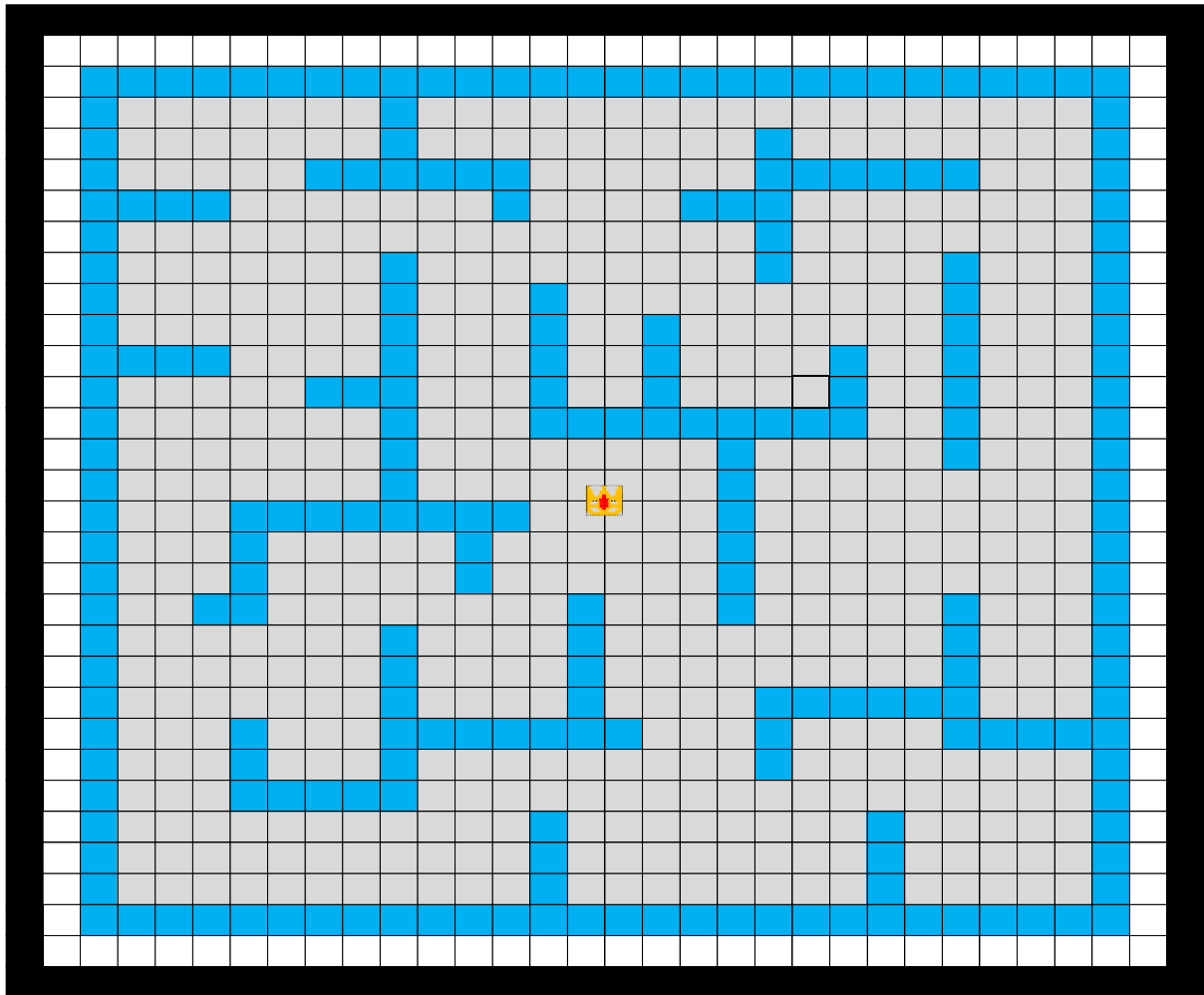


Working:

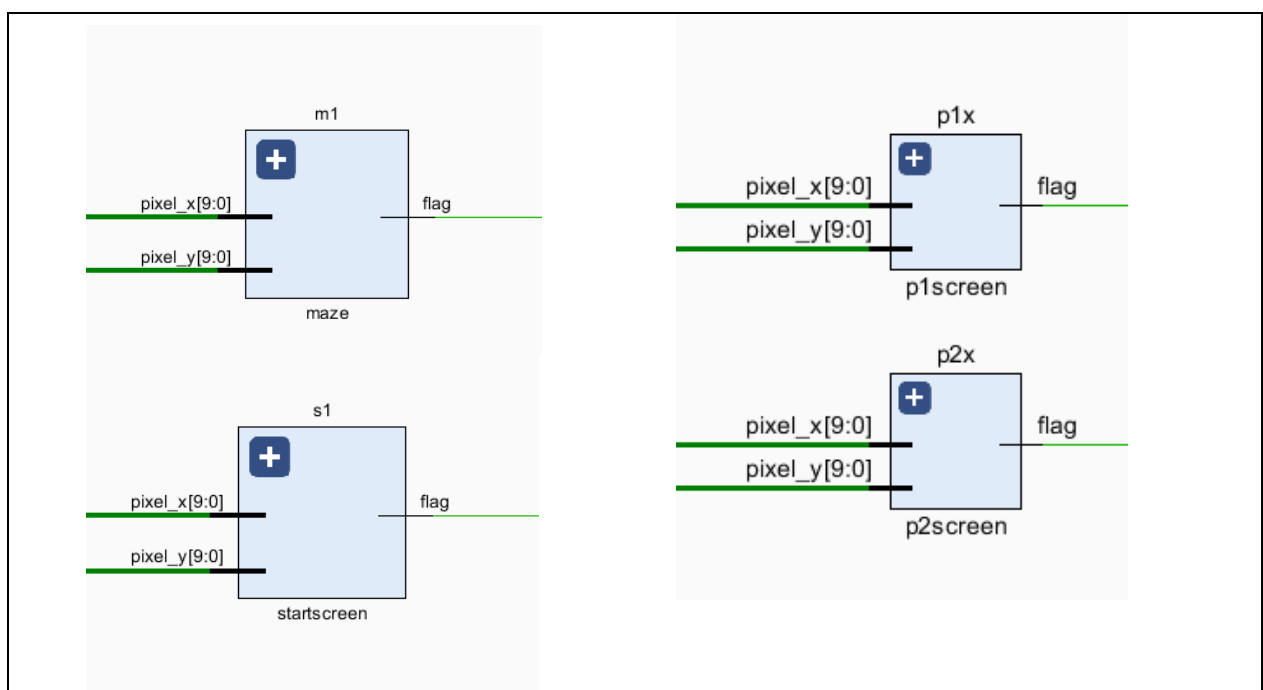
This module implements the main game FSM. It takes in key inputs from the keyboard module, and changes the state of the game based on the keys and collisions, and passes the tank control keys to the tank modules. It gets `pixel_x` and `pixel_y` from `vga_sync` module that tells it which pixel on the screen is being currently displayed. It passes these into the maze, tank1, tank2, startscreen, and winning screens modules, which tell it if these particular things are to be displayed on this pixel. For example, if the pixel is at a part where tank is, the tank will return flag true, and `pixel_gen` will return the color of the particular tank to the top.

Maze Module, Start & End Screen:

The map size is 30x30 boxes where each box has size of 16x16 pixels. It is handled by a 30x30 bitmap.



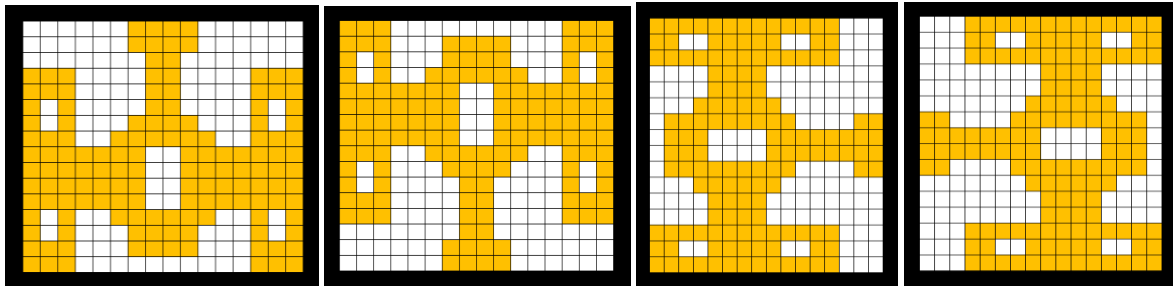
Block Diagram:



Working:

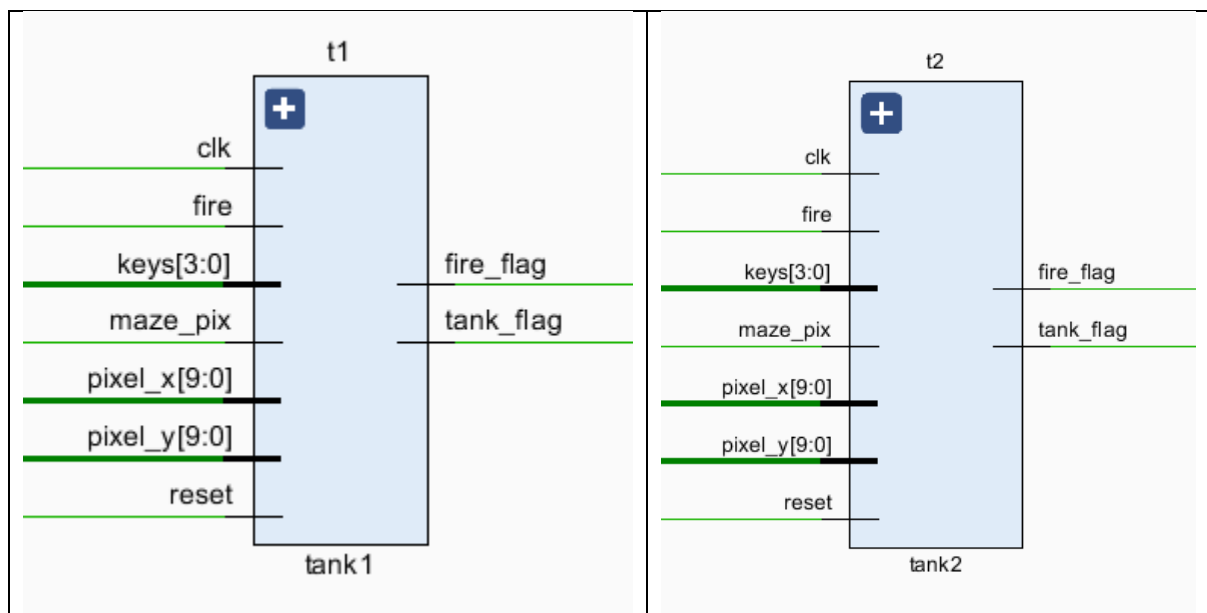
Maze module contains a 2-dimensional vector that stores a 30 x 30 bitmap of the map, which is scale by a ratio of 1:16 to be displayed on the screen. It returns a flag to tell pixel_gen whether to display maze on the current pixel position. The other three modules work the same way, to display pixelated text.

Tank Module:



The tanks are represented in a 16x16 pixel bitmaps, with separate bitmap for each direction.

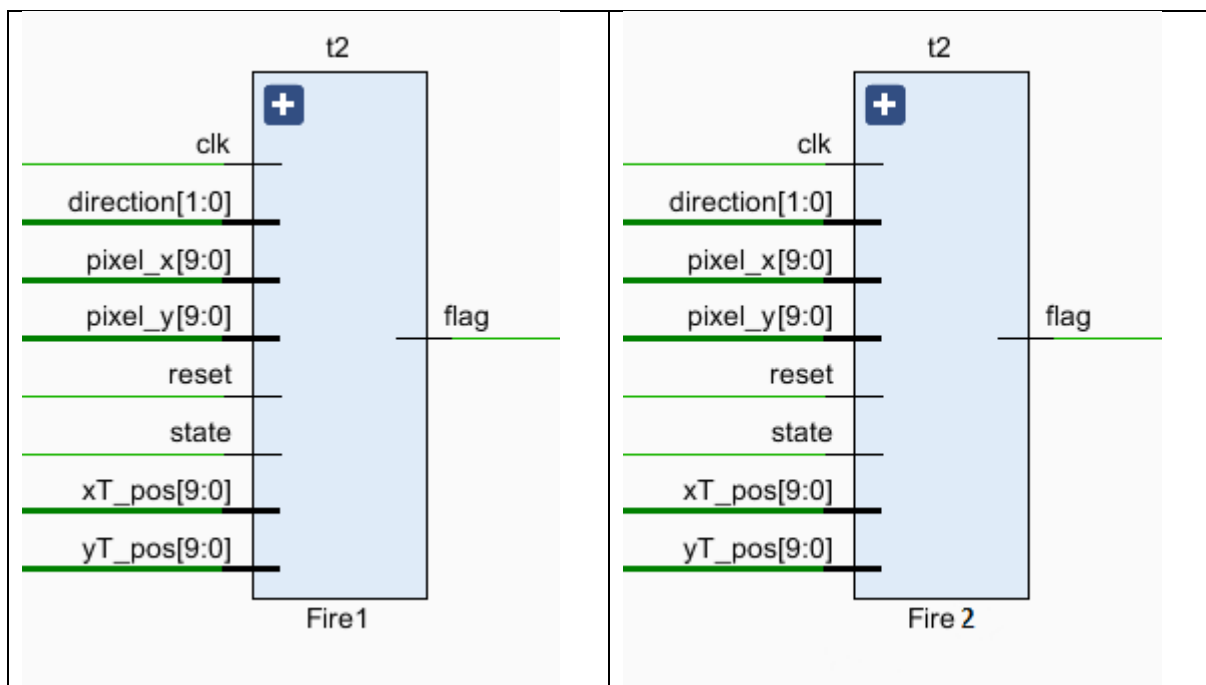
Block Diagram:



Working:

This module displays and controls the each tank for each player. This implements movement with collision FSM using key inputs passed by pixel_gen for movement, and maze, crown and the other tank's pixel for collision detection. It detects collisions by checking if there is a pixel of one of these three objects on each side of the tank, and uses this as input to collision FSM. The bitmap is split into 4 parts, with sprite for tank for each direction, and the one displayed is based on the current state. Upon reset flag from pixel_gen, the position and direction of the tank is reset. Each tank also has an instance of the fire module, which it passes the fire flag to generate fire.

Fire Module:

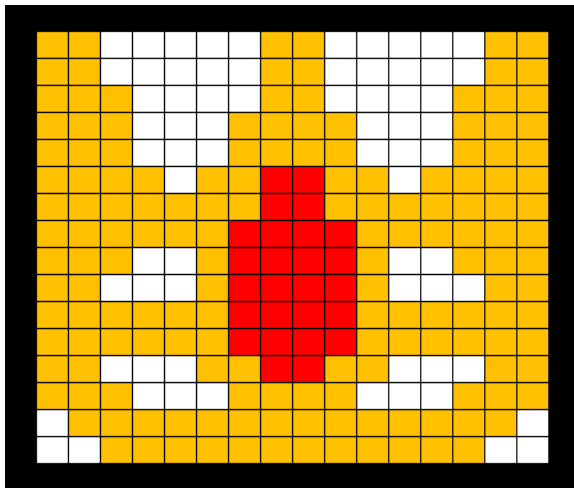


Working:

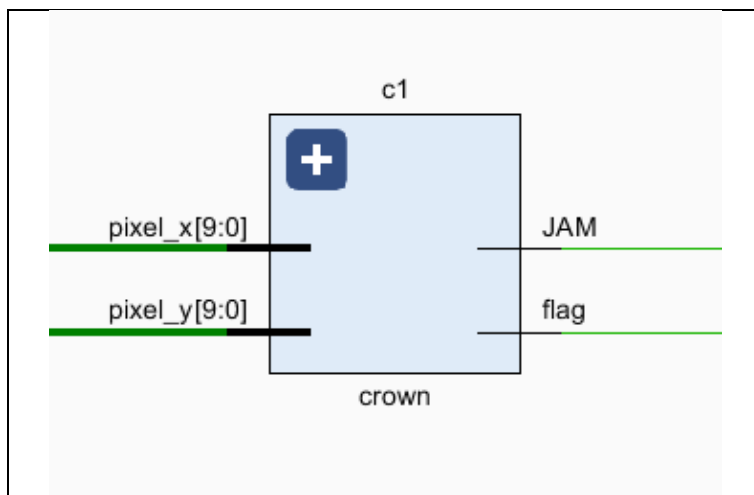
Each tank has an instance of the Fire1 module. This module shows the fire for each tank. It is passed the position and direction state of its tank and a flag for fire. When the fire flag is true, the display of the fire is set to true, and position is set relevant to position of tank (middle of tank), from where it is fired in the direction of the tank with a speed of 3pixels/frame. It is reset when it gets a true reset flag, which happens when either the tank module calling it, detects its collision with an object, or when the game is started.

Crown Module:

The crown is also represented in a 16x16 pixel bitmap.



Block Diagram:



Working:

This has bitmap vectors for crown and JAM, which it sends to pixel_gen. The crown is displayed at the center of screen. The JAM is displayed according to a timer, which counts till 750 frames, and the JAM is only displayed when frames are between 600 and 750 frames.

Conclusion & Results: Milestone 1

We implemented the movement of the tank in the maze till Milestone 1. We had worked out the input peripherals using keyboard by which we were able to take 12 different input keys from the keyboard. We were able to display our maze which consists of 30x30 blocks of dimension 16x16 pixels each through our pixel gen and VGA controller. We were also able to generate our tanks, consisting of 16x16 pixels, and move them according to the finite state machine we built to handle both direction of the tanks' orientation and the direction they were moving in. We had yet to implement collision of the tanks with the walls and the projectile fire to be launched from the tanks along with its collision with the walls and the other tanks.

Conclusion & Results: Milestone 2

We were able to implement the collision of the tanks with the walls by using the flag of the maze and the pixels of the tank depending on which side of the tank was colliding with the wall. Through the same logic we were also able to extend our collision to accommodate collision with the other tanks and the collectibles. We then worked to implement the projectile to only be launched when the fire button was pressed and hitting the other tank would end the game. After this we also worked on making the start and end screens using the same logic, we used to generate the maze and encompassing it in the screen state machine. To add more complexity to the game and make the game more challenging we added a crown which would only be collectible when it had the gem, timed to appear for a short time. The most challenging feature of the game was projectile generation on a button press and collision according to the different orientation of the tank and the collided side of the tank. We can add in more coins in the game to make the game even more interesting which would work on scores and we can also add in multiple lives for the players. In the future, we plan to complete the hardware prospectus of the game by making a physical arcade machine casing for the hardware to sit in along with using the joysticks and push buttons to give a proper feel of playing a retro style arcade game.

START
GAME!

